

# SSN College of Engineering – Kalavakkam

## Department of Computer Science and Engineering

### ICS1513 – Computer Networks Laboratory

#### Experiment 2: Student Record Retrieval Using TCP Socket

**Name:** Saravana Senthilkumar | **Roll No:** 3122247001057 | **Date:** 25.06.2026

## Objective

To implement a connection-oriented client-server application using TCP sockets in C where the client queries a student's record by sending a roll number, and the server searches a CSV file to return the matching record.

## Server Algorithm

1. Create a TCP socket using `socket(AF_INET, SOCK_STREAM, 0)`.
2. Define the server address structure (`AF_INET`, `INADDR_ANY`, Port 9000).
3. Bind the server socket to the specified IP address and port using `bind()`.
4. Put the server into passive listening mode with `listen()`.
5. Enter an infinite loop to accept incoming client connections using `accept()`.
6. Read the roll number string sent by the client using `read()`.
7. Call `find_student()` to open `students.csv`, skip the header row, and check line-by-line for a matching roll number.
8. If found, copy the full CSV line to the reply buffer; otherwise, populate the reply buffer with "Student not found\n".
9. Send the reply string back to the client using `write()`.
10. Close the active client connection socket with `close()` and continue waiting for the next connection.

## Client Algorithm

1. Create a TCP stream socket using `socket(AF_INET, SOCK_STREAM, 0)`.
2. Configure the server address structure with `AF_INET`, target port 9000, and convert IP string "127.0.0.1" using `inet_pton()`.
3. Establish a connection to the server using `connect()`.
4. Prompt the user to enter a roll number, read it via `fgets()`, and remove the trailing newline.
5. Send the roll number to the server using `write()`.
6. Read the server's response using `read()` and null-terminate the received byte stream.

7. Print the server response to standard output.
8. Close the socket using close().

## Server Code (tcp\_student\_server.c)

```
#include <stdio.h>
#include <string.h>
#include <arpa/inet.h>
#include <unistd.h>

// Search students.csv for the given roll number.
// If found, fill 'result' with the matching line and return 1.
// Otherwise return 0.
int find_student(char *roll, char *result) {
    FILE *fp = fopen("students.csv", "r");
    char line[256];

    if (!fp) return 0;

    fgets(line, sizeof(line), fp); // skip header line

    while (fgets(line, sizeof(line), fp)) {
        char line_copy[256];
        strcpy(line_copy, line);

        char *token = strtok(line_copy, ",");
        if (token != NULL && strcmp(token, roll) == 0) {
            strcpy(result, line);
            fclose(fp);
            return 1;
        }
    }

    fclose(fp);
    return 0;
}

int main() {
    int server_fd, client_fd;
    struct sockaddr_in addr;
    char roll[64], result[256], reply[300];

    server_fd = socket(AF_INET, SOCK_STREAM, 0);
```

```

addr.sin_family = AF_INET;
addr.sin_addr.s_addr = INADDR_ANY;
addr.sin_port = htons(9000);

bind(server_fd, (struct sockaddr*)&addr, sizeof(addr));
listen(server_fd, 1);

printf("TCP student server listening on port 9000...\n");

while (1) {
    client_fd = accept(server_fd, NULL, NULL);

    int n = read(client_fd, roll, sizeof(roll) - 1);
    if (n > 0) {
        roll[n] = '\0';
        printf("Client requested roll no: %s\n", roll);

        if (find_student(roll, result)) {
            strcpy(reply, result);
        } else {
            strcpy(reply, "Student not found\n");
        }

        write(client_fd, reply, strlen(reply));
    }
    close(client_fd);
}

close(server_fd);
return 0;
}

```

## Client Code (tcp\_student\_client.c)

```

#include <stdio.h>
#include <string.h>
#include <arpa/inet.h>
#include <unistd.h>

int main() {
    int sock;

```

```

struct sockaddr_in addr;
char roll[64], buffer[300];

printf("Enter roll no: ");
fgets(roll, sizeof(roll), stdin);
roll[strcspn(roll, "\n")] = '\0'; // strip newline

sock = socket(AF_INET, SOCK_STREAM, 0);

addr.sin_family = AF_INET;
addr.sin_port = htons(9000);
inet_pton(AF_INET, "127.0.0.1", &addr.sin_addr);

connect(sock, (struct sockaddr*)&addr, sizeof(addr));

write(sock, roll, strlen(roll));

int n = read(sock, buffer, sizeof(buffer) - 1);
if (n > 0) {
    buffer[n] = '\0';
    printf("Server response: %s\n", buffer);
}

close(sock);
return 0;
}

```

## Code Output (Test Cases)

### Server Output Terminal:

```

TCP student server listening on port 9000...
Client requested roll no: 3122247001057
Client requested roll no: 3122247001999

```

### Client Output — Test Case 1 (Valid Record Found):

```

Enter roll no: 3122247001057
Server response: 3122247001057,Saravana Senthilkumar,CSE

```

### Client Output — Test Case 2 (Record Not Found):

Enter roll no: 3122247001999

Server response: Student not found

## **Learning Outcomes**

1. Gained hands-on experience with the TCP connection lifecycle using POSIX socket calls (socket, bind, listen, accept, connect).
2. Understood stream-oriented byte communication and how reliable socket streams can be utilized for client-server database retrieval applications.